



24-Hour Execution Plan: Disaster Resource Allocation Mesh (DRAM)

Track: Agents for Good **Core Narrative:** An offline-first, multi-agent triage system for "Aapda Mitra" volunteers in Dharavi, Mumbai, designed to overcome LLM cognitive failures and "selfish node" resource hoarding during monsoon floods.

The 3 Kaggle Capstone Requirements (How we hit them)

Based on your research document, here is exactly how we fulfill the Kaggle requirements in code:

1. Multi-Agent ADK System (The Cognitive Pipeline)

Your research points out that frontier LLMs fail at multi-step disaster tasks (The DORA Benchmark). We will build the exact multi-agent graph from your paper to fix this:

- **Node 1: Meta Intake Agent:** Receives noisy reports ("friend hurt, bus stuck") and parses strict JSON.
- **Node 2: Assessor Agent:** Evaluates if the request meets the strict triage rubric.
- **Node 3: Task Generator Agent:** Formulates the final allocation order.

2. Custom Agent Skill (The "Game-Theoretic Reputation" Skill)

We will write a custom Python function (Skill) that calculates the $\Gamma(A_i)$ accountability score. If a node (like a private clinic) has been hoarding supplies, the skill automatically rejects their request based on the "Rawlsian Maximin" invariant.

3. MCP Server / Security Gateway (The "Cadence Council")

We will implement a simulated local JSON database (acting as the District War Room inventory) that the agents query. We will wrap this in a strict "Zero-Trust Security Node" that acts as the "Gated Valve" from your research, ensuring LLMs cannot directly overwrite database state without passing a deterministic validation check.

Phase 1: Workspace Setup (Tonight - 30 Mins)

Create a new folder named `dram-agent` and initialize the project:

```
mkdir dram-agent
cd dram-agent
uv init
uv add google-genai agents-cli pytest pydantic
```

Folder Structure to Create:

```

dram-agent/
├── agents/
│   ├── intake_agent.py      # Meta Node
│   ├── assessor_agent.py    # Triage Node
│   └── allocation_agent.py   # Task Generator
├── skills/
│   └── reputation_calc.py    # Computes Invariant I (Selfish Node Penalty)
├── db/
│   └── dharavi_inventory.json # Simulated District War Room state
├── tests/
│   └── test_mesh.py          # Pytest for offline sync limits
└── main.py                   # ADK Graph execution

```



Phase 2: The "Vibe Coding" Prompts (Tonight - 2 Hours)

You don't need to write this from scratch. Open **Cursor**, **Windsurf**, or **Copilot**, point it at your new directory, and paste these exact prompts one by one:

Prompt 1: The Database & Skill

"Create a local JSON file simulating a disaster inventory in Dharavi (oxygen, med-kits). Then, write a Python class using Google ADK tool syntax called `ReputationSkill`. It takes a `node_id`, reads their past behavior, and calculates a penalty score based on the Rawlsian Maximin principle. If the score is below 0.5, it returns a block action."

Prompt 2: The Multi-Agent Graph

"Using the Google ADK graph topology, build a 3-node stateful workflow. Node 1 (Intake) takes a noisy text string and extracts Location, Urgency, and Request. Node 2 (Assessor) calls the `ReputationSkill`. If the node is selfish, it terminates. If not, it passes the context to Node 3. Node 3 (Allocator) deducts the requested items from the local JSON inventory and outputs a final action plan. Use Pydantic for state management."

Prompt 3: The Security / Testing Layer

"Write a `pytest` suite that simulates three scenarios: 1. A generous Aapda Mitra volunteer requesting 1 kit (Passes). 2. A selfish private hospital requesting 50 oxygen tanks with a bad reputation (Blocked by invariant). 3. A prompt injection attempt in the intake string (Caught by Pydantic validation)."



Phase 3: The Video Demo Script (Tomorrow Morning)

Kaggle requires a <5 minute video. You don't need UI. Terminal output is actually better for this highly technical concept.

Script Outline (3 Minutes):

- **0:00-0:45 (The Hook):** "Traditional AI fails in disasters due to the 'generous-agent bias' and LLM cognitive collapse. For my Capstone, I built the Disaster Resource Allocation Mesh (DRAM) tailored for Dharavi's Aapda Mitra volunteers."

- **0:45-1:30 (Architecture):** Show your code. Explain how you bypassed the DORA benchmark failures by splitting the LLM into a 3-Node ADK Graph (Intake -> Assessor -> Allocator).
- **1:30-2:30 (The Live Demo):** Run the terminal. Show a successful allocation. Then, show the system mathematically rejecting a "selfish node" trying to hoard resources using your custom Reputation Skill.
- **2:30-3:00 (Conclusion):** Emphasize that all database writes pass through a deterministic security validation (simulating a hardware air-gap) to guarantee Zero-Trust.



Phase 4: Final Submission (Tomorrow Afternoon)

1. Run your `pytest` suite to ensure the terminal output looks green and clean.
2. Push everything to GitHub with a stunning `README.md` containing the architecture diagram from your research.
3. Upload the video to YouTube (Unlisted).
4. Submit the Kaggle form by tomorrow evening, and go get ready for your first day of classes!